Ch 08: String Slicing

Application Program Development

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2026



Basic String Operations

Strings

Python provides several ways to access the individual characters in a string. Strings also have methods that allow you to perform operations on them.

- Many types of programs perform operations on strings
- In Python, many tools for examining and manipulating strings
  - Strings are sequences, so many of the tools that work with sequences work with strings

EAST TEXAS A&M

Accessing the Individual Characters in a String (1 of 3)

- Use a `for` loop
  - Format: `for character in my_string:`
  - Useful when need to iterate over the whole string, such as count occurrences of a specific character

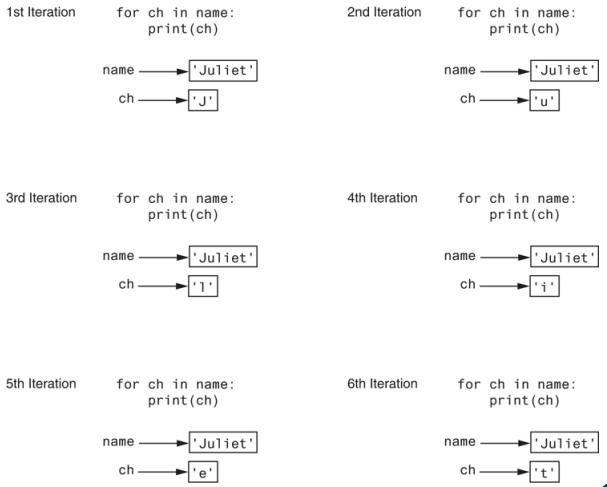


Figure 1: Iterating over the string Juliet (Gaddis, 2021, pg 455)

Accessing the Individual Characters in a String (2 of 3)

- Use indexing
  - Each character has an index (like a tuple or list sequence) specifying its position
  - 0 based indexing, as for all Python sequences
  - Format: `character = my_string[6]`

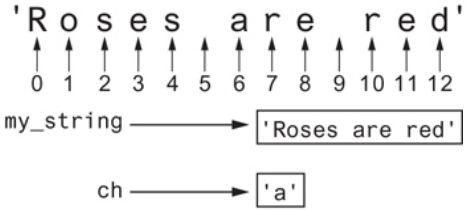


Figure 2: String indexes (Gaddis, 2021, pg.457)

EAST TEXAS A&M

Accessing the Individual Characters in a String (3 of 3)

- `IndexError` exception will occur if:
  - You try to use an index that is out of range for the string
  - Likely to happen when loop iterates beyond the end of the string
  - Of course this is true of any sequence access using too large of an index
- `len(string)` function can be used to obtain the length of a string
  - Useful to prevent loops from iterating beyond the end of a string

EAST TEXAS A&M

String Concatenation

- **Concatenation**: appending one string to the end of another string
  - Use the + operator to produce a string that is a combination of its operands
  - The augmented assignment operator += can also be used to concatenate strings
    - The operand on the left side of the += operator must be an existing variable, otherwise an exception is raised



Strings are Immutable

- Strings are an immutable sequence (like tuple)
  - Once they are created, they cannot be changed
    - Concatenation doesn't actually change the existing string, but rather creates a new string and assigns the new string to the previously used variable
  - Cannot use an expression of the form
  - `string[index] = new_character`
    - Statement of this type will raise an exception

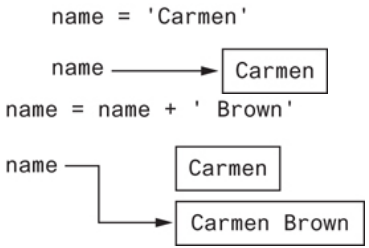


Figure 3: The string Carmen Brown from concatenation actually creates a new string (Gaddis, 2021, pg.460)

EAST TEXAS A&M

String Slicing

- **Slice**: span of items taken from a sequence, known as *substring*
  - Slicing format: `string[start : end]`
    - Expression will return a (new) string containing a copy of the characters from `start` up to but not including `end`
    - If `start` not specified, 0 is used for `start` index
    - If `end` not specified, len(string) is used for `end` index
  - Slicing expressions can include a step value and negative indexes relative to end of string
    - Same as list slicing we covered previously



EAST TEXAS A&M

Summary

- `string` types in Python are immutable sequences
  - So many of the tools and syntax we discussed about sequences (like lists and tuples) apply
- Can iterate over the characters of a string with a `for` loop
- Can access characters in a string sequence using integer index
- Strings can be concatenated using the + operator
- However, strings are an immutable type, for example you can't use index into a string to change a character in the string
- String slicing also works as we saw for other sequence types, `string[start : end]`

EAST TEXAS A&M

References

Gaddis, T. (2021). *Starting out with Python* (5th). Pearson.



EAST TEXAS A&M